ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO HỌC PHẦN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

ĐỀ TÀI: HỆ THỐNG QUẢN LÝ MUA VÉ XEM PHIM CHO SINH VIÊN

Giảng viên hướng dẫn: ThS.Trần Đình Nam Sơn

Lớp tín chỉ: CSE703029-2-3-25(N01)

Nhóm thực hiện: Nhóm 06

Thành viên thực hiện:

1. **Nguyễn Hữu Lan - 24103589**
2. **Trần Quốc Cường - 24106195**
3. **Đào Quang Huy - 23010835**
4. **Nguyễn Văn Duy - 24100555**
5. **Nguyễn Quang Long - 24108169**
6. **Đỗ Đức Khoa - 24100223**

Hà Nội, tháng 7 năm 2026

BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM

STT	Họ và tên	Tên github	Nhánh github	Vai trò	Công việc phụ trách
1	Nguyễn Hữu Lan	nguyenhuulan297	Nguyễn-Hữu-Lan-24103589	Nhóm trưởng	Thiết kế kiến trúc hệ thống, xây dựng nền tảng chung của dự án, phát triển module Booking/Ticket, tích hợp các module, thiết kế giao diện và tổng hợp báo cáo.
2	Trần Quốc Cường	cuongdeptraisieucapvutruvjppro	Trần-Quốc-Cường-24106195	Thành viên	Phát triển module Customer & Price Policy, quản lý khách hàng và xây dựng chính sách tính giá vé theo từng loại khách hàng.
3	Đào Quang Huy	huybibeo	Đào-Quang-Huy-23010835	Thành viên	Phát triển module Movie, quản lý danh sách phim, tìm kiếm phim và xử lý các chức năng liên quan đến thông tin phim.
4	Nguyễn Văn Duy	ndyXx-noob	Nguyễn-Văn-Duy-24100555	Thành viên	Phát triển module Showtime, quản lý suất chiếu, liên kết thông tin phim với phòng chiếu và lịch chiếu.
5	Nguyễn Quang Long	nglong3011	Nguyễn-Quang-Long-24108169	Thành viên	Phát triển module Seat & CinemaRoom, quản lý ghế ngồi, phòng chiếu và chức năng đặt ghế.
6	Đỗ Đức Khoa	khoadevieXLM	Đỗ-Đức-Khoa-24100223	Thành viên	Phát triển module Payment, xử lý thanh toán và cập nhật trạng thái vé sau khi thanh toán.

BẢNG ĐÁNH GIÁ MỨC ĐỘ HOÀN THÀNH CÔNG VIỆC VÀ NHẬN XÉT THÀNH VIÊN

STT	Họ và tên	MSSV	Vai trò	Đánh giá	Nhận xét
1	Nguyễn Hữu Lan	24103589	Nhóm trưởng	100%	
2	Trần Quốc Cường	24106195	Thành viên	100%	Thái độ làm việc nghiêm túc, tinh thần trách nhiệm cao. Luôn chủ động, hoàn thành 100% yêu cầu đúng thời hạn và phối hợp rất tốt với nhóm.
3	Đào Quang Huy	23010835	Thành viên	100%	Nhiệt tình, có tinh thần tập thể tốt. Nộp sản phẩm đúng cam kết, sẵn sàng tiếp thu góp ý và đóng góp tích cực vào kết quả chung.
4	Nguyễn Văn Duy	24100555	Thành viên	100%	Làm việc cẩn thận, tuân thủ đúng tiến độ đề ra. Thái độ hợp tác tích cực, hòa đồng và trách nhiệm trong suốt quá trình làm việc nhóm.
5	Nguyễn Quang Long	24108169	Thành viên	100%	Luôn hoàn thành xuất sắc nhiệm vụ đúng hạn với thái độ chủ động. Tương tác tốt với các thành viên và có tinh thần đồng đội cao.
6	Đỗ Đức Khoa	24100223	Thành viên	70%	Phần việc cá nhân chưa thật sự chỉn chu và chỉ hoàn thành khoảng 70%.

Mục lục

CHƯƠNG 1: GIỚI THIỆU DỰ ÁN

1.1 Tổng quan

1.2 Mục tiêu dự án

1.3 Phạm vi hệ thống

CHƯƠNG 2: CÔNG NGHỆ VÀ KIẾN TRÚC HỆ THỐNG

2.1 Công nghệ sử dụng

2.2 Kiến trúc hệ thống

2.3 Cấu trúc package Backend

2.4 Luồng xử lý request

2.5 Sơ đồ class diagram

CHƯƠNG 3: PHÂN TÍCH LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

3.1 Đóng gói (Encapsulation)

3.2 Kế thừa (Inheritance)

3.3 Đa hình (Polymorphism)

3.4 Trừu tượng (Abstraction)

3.5 Tổng hợp áp dụng OOP

CHƯƠNG 4: MÔ TẢ TÍNH NĂNG HỆ THỐNG

4.1 Xác thực người dùng

4.2 Quản lý phim

4.3 Quản lý suất chiếu

4.4 Quản lý phòng chiếu

4.5 Quản lý ghế

4.6 Đặt vé

4.7 Thanh toán

4.8 Vé của tôi & hồ sơ khách hàng

CHƯƠNG 5: LUỒNG NGHIỆP VỤ ĐẶT VÉ XEM PHIM

5.1 Luồng chính: từ đăng nhập đến thanh toán

5.2 Xử lý tình huống ghế bị tranh chấp

5.3 Xử lý tình huống thanh toán thất bại

CHƯƠNG 6: BẢO MẬT VÀ KỸ THUẬT

6.1 Xác thực và bảo vệ mật khẩu

6.2 CORS

6.3 Xử lý ngoại lệ và mã lỗi HTTP

6.4 Lưu trữ dữ liệu bằng file JSON

6.5 Sinh mã định danh (ID)

6.6 Kiểm thử tự động (Unit Test)

CHƯƠNG 7: TRIỂN KHAI VÀ VẬN HÀNH

7.1 Yêu cầu môi trường

7.2 Lệnh chạy hệ thống

7.3 Tài khoản dùng thử
7.4 Giới hạn khi vận hành thực tế

CHƯƠNG 8: KIỂM THỬ VÀ KỊCH BẢN TEST

8.1 Mục tiêu kiểm thử
8.2 Môi trường và công cụ kiểm thử
8.3 Các tình huống test
8.3.1 TC01 – Đặt vé thành công cho khách hàng thường
8.3.2 TC02 – Đặt vé cho ghế đã được đặt (xung đột)
8.3.3 TC03 – Chọn ghế không tồn tại
8.3.4 TC04 – Sinh viên đặt vé được giảm giá 20%
8.3.5 TC05 – Đặt vé cho phim đã ngừng chiếu
8.3.6 TC06 – Thanh toán thành công cho vé đã đặt
8.4 Tổng kết kết quả kiểm thử

CHƯƠNG 9: KẾT LUẬN

9.1 Kết quả đạt được
9.2 Điểm mạnh của dự án
9.3 Hạn chế và hướng phát triển

CHƯƠNG 1: GIỚI THIỆU DỰ ÁN

1.1. Tổng quan

Hệ thống Quản lý Mua vé Xem phim (Student Movie Ticket Management System) là bài tập lớn học phần **Lập trình Hướng đối tượng** của Nhóm 06, xây dựng dưới dạng một ứng dụng web đặt vé xem phim hoàn chỉnh: người dùng đăng ký/đăng nhập tài khoản, xem danh sách phim đang chiếu, chọn suất chiếu, chọn ghế trên sơ đồ phòng chiếu, đặt nhiều vé cùng lúc và thanh toán bằng một trong ba phương thức được hỗ trợ.

Dự án gồm hai thành phần tách biệt, giao tiếp với nhau qua REST API:

- **Backend**: xây dựng bằng Java 21 và Spring Boot 4.1.0 (module `spring-boot-starter-webmvc`), cung cấp 22 endpoint REST cho toàn bộ nghiệp vụ đặt vé. Dữ liệu nghiệp vụ (phim, phòng chiếu, suất chiếu, ghế, khách hàng, vé, giao dịch thanh toán) được lưu trữ dưới dạng các file JSON trong thư mục `backend/data/`, không dùng hệ quản trị cơ sở dữ liệu thật — đúng như README của dự án ghi rõ: "chỉ phục vụ mục đích học tập/demo".
- **Frontend**: xây dựng bằng React 18 + TypeScript, dùng Vite làm build tool, Tailwind CSS cho giao diện, React Router cho điều hướng và Axios để gọi API.

1.2. Mục tiêu dự án

1. **Số hóa quy trình đặt vé xem phim**: xây dựng luồng nghiệp vụ hoàn chỉnh từ chọn phim, chọn suất chiếu, chọn ghế, đặt vé đến thanh toán.
2. **Áp dụng 4 tính chất OOP**: đóng gói, kế thừa, đa hình, trừu tượng được thể hiện rõ ràng qua các lớp thực thể (model), chính sách giá vé (price policy) và phương thức thanh toán (payment method).
3. **Kiến trúc phân tầng rõ ràng**: Handler (Controller) → Service → Repository, tách biệt trách nhiệm giữa tầng tiếp nhận HTTP, tầng xử lý nghiệp vụ và tầng truy xuất dữ liệu.
4. **Áp dụng các mẫu thiết kế (design pattern) cơ bản**: Factory Method (`CustomerFactory`), Strategy (chính sách giá vé, phương thức thanh toán) để minh họa tính đa hình trong tình huống thực tế.
5. **Bảo vệ dữ liệu người dùng ở mức cơ bản**: mật khẩu không bao giờ được lưu ở dạng plaintext hay trả về cho client, luôn đi qua bước băm (hash) trước khi lưu.

1.3. Phạm vi hệ thống

Khác với các hệ thống quản lý đa vai trò (nhân viên, quản trị viên...), phạm vi của dự án tập trung vào một vai trò duy nhất là **khách hàng (Customer)**, nhưng khách hàng lại được chia thành 3 loại — mỗi loại có chính sách giá vé riêng:

STT	Loại khách hàng	Enum	Ưu đãi giá vé	Thuộc tính riêng
1	Khách thường	`NORMAL`	Không giảm giá	—
2	Sinh viên	`STUDENT`	Giảm 20%	`studentId`, `schoolName`
3	Thành viên VIP	`VIP`	Giảm 30%	`memberSince`, `pointBalance`

Ngoài nghiệp vụ đặt vé dành cho khách hàng, hệ thống còn cung cấp các API quản trị danh mục dùng chung (phim, suất chiếu, phòng chiếu) để phục vụ việc khởi tạo/vận hành dữ liệu nền cho rạp — trong đó module **phòng chiếu (Cinema Room)** có đầy đủ thao tác CRUD (tạo/sửa/xoá/xem).

CHƯƠNG 2: CÔNG NGHỆ VÀ KIẾN TRÚC HỆ THỐNG

2.1. Công nghệ sử dụng

2.1.1. Backend

Công nghệ	Vai trò / Mục đích
Java 21	Ngôn ngữ lập trình chính, hỗ trợ đầy đủ OOP (lớp trừu tượng, interface, kế thừa).
Spring Boot 4.1.0	Framework ứng dụng web: IoC/DI container, cấu hình tự động.
`spring-bootstarter-webmvc`	Xây dựng REST API (`@RestController`), ánh xạ HTTP request/response. Nội bộ kéo theo `springboot-starter-jackson` để tự động chuyển đổi object Java ⇄ JSON cho phần thân (body) của response.
Gson 2.13.1	Đọc/ghi dữ liệu nghiệp vụ ra file JSON ở tầng Repository (khác với Jackson dùng cho HTTP response — xem mục 2.4).
Maven (mvnw)	Quản lý dependency, build project, chạy test.
JUnit 5 + Mockito	Viết unit test cho model, repository, service (qua `spring-boot-starter-webmvc-test`).

2.1.2. Frontend

Công nghệ	Vai trò / Mục đích
React 18 + TypeScript	Framework UI, kiểu dữ liệu tĩnh, kiến trúc component.
Vite	Build tool / dev server cho dự án React + TypeScript.
React Router DOM v6	Điều hướng phía client, `ProtectedRoute` chặn truy cập khi chưa đăng nhập.
Axios	HTTP client gọi REST API của backend (`http://localhost:8080/api`).
Tailwind CSS v4	Styling utility-first cho giao diện đặt vé, sơ đồ ghế.
Context API	`AuthContext` lưu khách hàng đã đăng nhập (localStorage), `ToastContext` hiển thị thông báo.

2.2. Kiến trúc hệ thống

Hệ thống được tổ chức theo kiến trúc phân tầng (Layered Architecture), chia thành 3 tầng chính ở phía backend cộng với tầng lưu trữ file:

> Tầng 1 — Handler (Controller): tiếp nhận HTTP request, chuyển đổi thân request thành DTO nội bộ (các lớp `static class ...Request` khai báo ngay trong từng Handler), gọi Service tương ứng, bắt các exception nghiệp vụ cụ thể và ánh xạ sang đúng mã HTTP status.
>
> Tầng 2 — Service: xử lý toàn bộ logic nghiệp vụ (kiểm tra hợp lệ, tính giá vé, xử lý thanh toán, điều phối giữa nhiều repository), ném ra ngoại lệ tuỳ biến khi vi phạm quy tắc nghiệp vụ.
>
> Tầng 3 — Repository: trừu tượng hoá thao tác đọc/ghi dữ liệu; mỗi repository chịu trách nhiệm cho một loại thực thể, đọc/ghi vào đúng một file JSON tương ứng trong `backend/data/` .
>
> Tầng 4 — File JSON: thay cho cơ sở dữ liệu thật, dữ liệu được lưu trực tiếp dưới dạng mảng JSON trên đĩa, đọc/ghi toàn bộ file mỗi lần truy vấn.

2.3. Cấu trúc package Backend

Khác với cách tổ chức theo domain nghiệp vụ, project này tổ chức package theo **layer (tầng)** — toàn bộ mã nguồn nằm trong `backend/src/main/java/` :

```
config/            <- Cấu hình Spring
(CORS)   └─ WebConfig.java
exception/          <- 9 lớp ngoại lệ tuỳ biến (đều extends
RuntimeException)   ├─ AuthenticationException.java
  ├─ CustomerNotFoundException.java
  ├─ EmailAlreadyExistsException.java
  ├─ MovieNotShowingException.java
  ├─ PaymentFailedException.java
  ├─ SeatAlreadyBookedException.java
  ├─ SeatNotFoundException.java
  ├─
ShowtimeNotFoundException.java
└─ TicketNotFoundException.java
handler/            <- 9 @RestController (tiếp nhận HTTP, trả HTTP
response)   ├─ AuthHandler, BookingHandler, CinemaRoomHandler,
CustomerHandler   ├─ MovieHandler, PaymentHandler, SeatHandler,
ShowtimeHandler, TicketHandler model/              <- 12 lớp thực thể + 6
enum   ├─ User (abstract), Customer (abstract extends User)
  ├─ NormalCustomer, StudentCustomer, VIPCustomer (extends Customer)
  ├─ Movie, CinemaRoom, Showtime, Seat, Booking, Ticket, Payment   └─
CustomerType, MovieStatus, SeatType, SeatStatus, PaymentStatus, PaymentMethodType
repository/         <- 7 @Repository + 1 adapter nội bộ   ├─ CustomerRepository,
MovieRepository, ShowtimeRepository, SeatRepository
  ├─ TicketRepository, PaymentRepository, CinemaRoomRepository   └─
CustomerTypeAdapter (Gson JsonSerializer/JsonDeserializer cho Customer đa hình)
service/            <- Business logic + Strategy pattern
  ├─ AuthService, BookingService, CustomerService, MovieService
  ├─ ShowtimeService, SeatService, CinemaRoomService, PaymentService
  ├─ CustomerFactory (Factory Method tạo đúng subclass Customer)
  ├─ ITicketPricePolicy +
NormalCustomerPricePolicy/StudentPricePolicy/VIPPricePolicy   └─ IPaymentMethod +
CashPayment/BankTransferPayment/EWalletPayment utils/              <- Tiện ích
dùng chung   ├─ FileUtils (đọc/ghi file JSON)
  ├─ IdGenerator (sinh mã ID)
  ├─ PasswordUtils (băm mật khẩu SHA-256 + salt)
  └─ Validator (kiểm tra dữ liệu đầu vào)
```

Cấu trúc package chia theo tầng (layer-first), khác với cách chia theo domain nghiệp vụ.

2.4. Luồng xử lý request

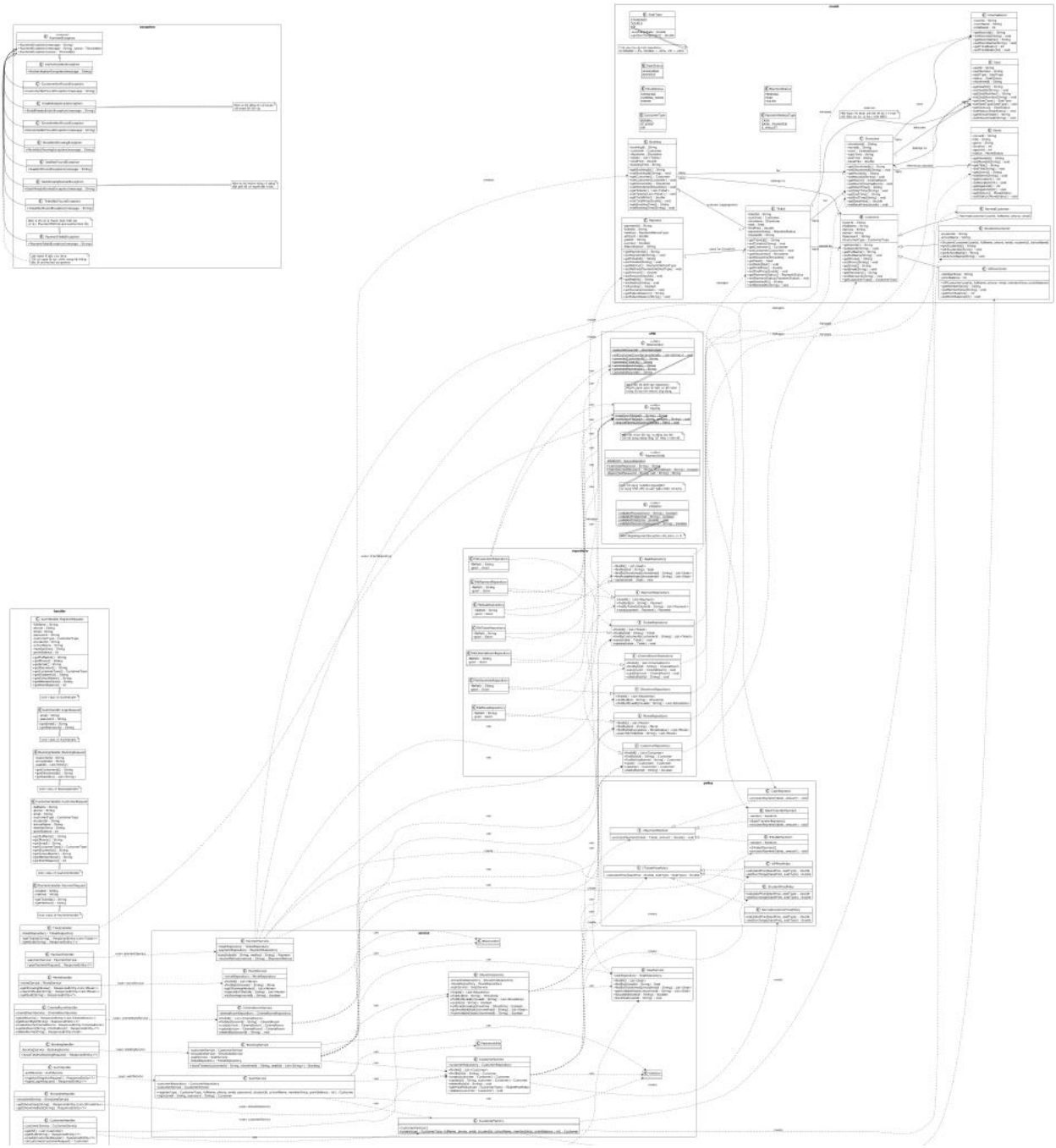
Một request điển hình từ Frontend đi qua các bước sau:

1. **Client** (trang React, qua Axios) gửi HTTP request tới `http://localhost:8080/api/...` .
2. `WebConfig` (Spring MVC CORS) kiểm tra origin — chỉ chấp nhận `http://localhost:3000` và `http://localhost:3001` , với các phương thức GET/POST/PUT/DELETE.
3. **Handler** (`@RestController`) nhận request, deserialize thân request (JSON) thành DTO nội bộ nhờ **Jackson** (tự động kèm theo bởi `spring-boot-starter-webmvc`), rồi gọi phương thức public của **Service** tương ứng.
4. **Service** xử lý nghiệp vụ: gọi các Service/Repository khác nếu cần, áp dụng chính sách giá vé hoặc phương thức thanh toán phù hợp (đa hình), ném ngoại lệ tuỳ biến (ví dụ `SeatAlreadyBookedException`) nếu vi phạm quy tắc.
5. **Repository** đọc/ghi file JSON tương ứng bằng **Gson** thông qua `FileUtils` — Gson ở đây chỉ phục vụ việc (de)serialize dữ liệu lưu trữ trên đĩa, hoàn toàn tách biệt với Jackson dùng cho tầng HTTP.

6. **Handler** bắt các exception nghiệp vụ cụ thể trong khối `try/catch` , ánh xạ sang đúng mã HTTP status (400/401/404/409/500) rồi trả về `ResponseEntity` . Khi thành công, Jackson tự động serialize object trả về (Customer, Booking, Ticket...) thành JSON gửi cho client.

> **Nhận xét:** dự án dùng song song hai bộ engine JSON khác nhau cho hai mục đích khác nhau — Jackson (mặc định của Spring MVC) lo phần request/response HTTP, còn Gson được chọn có chủ đích ở tầng Repository vì hỗ trợ đăng ký `TypeAdapter` tuỳ biến (`CustomerTypeAdapter`), cần thiết để đọc/ghi đúng subclass đa hình của `Customer` khi lưu xuống file JSON (xem mục 3.3).

2.5 Sơ đồ class diagram



CHƯƠNG 3: PHÂN TÍCH LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Dự án thể hiện đầy đủ 4 tính chất OOP thông qua các file code cụ thể, không chỉ dừng ở lý thuyết.

3.1. Đóng gói (Encapsulation)

Định nghĩa: che giấu dữ liệu và chi tiết cài đặt bên trong, chỉ cho phép tương tác qua các phương thức public.

3.1.1. Đóng gói dữ liệu ở các lớp model

Toàn bộ 12 lớp thực thể (`User` , `Customer` , `Movie` , `CinemaRoom` , `Showtime` , `Seat` , `Booking` , `Ticket` , `Payment` ...) đều khai báo field ở mức `private` và chỉ lộ ra bên ngoài qua cặp getter/setter public. Riêng trường mật khẩu của `User` được chú thích rõ quy ước lưu trữ ngay tại getter:

```java
public abstract class User {
    private String userId;
    private String fullName;
    private String phone;
    private String email;
    private String password;

    /** Lưu dạng đã băm (salt:hash), không bao giờ lưu plaintext. */
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

Listing 3.1: User.java — field private, giao tiếp qua getter/setter

3.1.2. Đóng gói logic xác thực trong AuthService

`AuthHandler` chỉ gọi `authService.login(email, password)` mà không cần biết bên trong có băm SHA-256, có so khớp salt như thế nào:

```java
public Customer login(String email, String password) {
    Customer customer = customerRepository.findByEmail(email);
    if (customer == null || !PasswordUtils.matches(password, customer.getPassword())) {
        throw new AuthenticationException("Email hoặc mật khẩu không đúng");
    }
    customer.setPassword(null); // không trả mật khẩu (dù đã băm) về cho client
    return customer;
}
```

Listing 3.2: AuthService.java — che giấu chi tiết băm mật khẩu và so khớp

Thông báo lỗi đăng nhập được gộp chung (“Email hoặc mật khẩu không đúng”) thay vì báo riêng “sai email” / “sai mật khẩu”, tránh lộ thông tin tài khoản nào tồn tại trong hệ thống — cũng là một hình thức đóng gói ở mức nghiệp vụ.

3.1.3. Đóng gói thuật toán băm mật khẩu trong PasswordUtils

```java
public class PasswordUtils {
    public static String hash(String rawPassword) {
        byte[] salt = new byte[16];
        RANDOM.nextBytes(salt);
        String saltB64 = Base64.getEncoder().encodeToString(salt);
        return saltB64 + ":" + digest(rawPassword, saltB64);
    }

    public static boolean matches(String rawPassword, String storedHash) {
        String[] parts = storedHash.split(":", 2);
        return digest(rawPassword, parts[0]).equals(parts[1]);
    }
    // digest(): SHA-256 – chi tiết thuật toán được giấu private, không lộ ra ngoài lớp
}
```

Listing 3.3: PasswordUtils.java — nơi duy nhất biết định dạng lưu trữ "salt:hash"

3.1.4. Đóng gói ở tầng Repository

Mỗi Repository che giấu hoàn toàn việc đọc/ghi file JSON phía sau các phương thức đơn giản (`findAll` , `findById` , `save` , `update`). Ví dụ, Service gọi `ticketRepository.save(ticket)` mà không biết dữ liệu được nối vào một `List<Ticket>` rồi ghi đè toàn bộ file `tickets.json` .

3.2. Kế thừa (Inheritance)

Định nghĩa: cho phép lớp con tái sử dụng thuộc tính/phương thức của lớp cha, tránh lặp code.

3.2.1. Chuỗi kế thừa User → Customer → {Normal, Student, VIP}Customer

`User` là lớp trừu tượng chứa các trường chung cho mọi tài khoản (userId, fullName, phone, email, password). `Customer` kế thừa `User` , bổ sung `customerType` , và bản thân cũng là lớp trừu tượng — buộc phải khởi tạo qua một trong 3 lớp con cụ thể:

```java
public abstract class Customer extends User {
    private CustomerType customerType;

    public Customer(String userId, String fullName, String phone, String email, CustomerType customerTy
pe) {
        super(userId, fullName, phone, email);    // tái sử dụng constructor của User
        this.customerType = customerType;
    }
}

public class StudentCustomer extends Customer {
    private String studentId;
    private String schoolName;

    public StudentCustomer( String userId, String fullName, String phone, String email,
                            String studentId, String schoolName) {
        super(userId, fullName, phone, email, CustomerType.STUDENT);
        this.studentId = studentId;
        this.schoolName = schoolName;
    }
}
```

Listing 3.4: Customer.java và StudentCustomer.java — kế thừa qua super(...)

Lớp con	Thuộc tính bổ sung riêng
`NormalCustomer`	Không có (chỉ gán `customerType = NORMAL`)
`StudentCustomer`	`studentId`, `schoolName`
`VIPCustomer`	`memberSince`, `pointBalance`

Cả 3 lớp con đều override `toString()` và gọi lại `super.toString()` để nối chuỗi mô tả kế thừa từ lớp cha, ví dụ: `"StudentCustomer{" + super.toString() + ", studentId=..." + "}"`.

3.2.2. Chuỗi kế thừa ngoại lệ nghiệp vụ

Toàn bộ 9 lớp ngoại lệ trong package `exception/` đều kế thừa trực tiếp `RuntimeException` (unchecked exception), mỗi lớp đại diện đúng một loại lỗi nghiệp vụ và chỉ định nghĩa constructor nhận `message`:

```
1    public class SeatAlreadyBookedException extends RuntimeException {
2    public SeatAlreadyBookedException(String message) { super(message); }
3    }
```

Listing 3.5: một trong 9 lớp ngoại lệ — mẫu hình lặp lại nhất quán nhờ kế thừa

Việc để tất cả cùng kế thừa `RuntimeException` giúp các Handler không bị bắt buộc khai báo `throws`, đồng thời vẫn có thể bắt riêng từng loại lỗi bằng `catch (Loại Cụ Thể e)` để ánh xạ đúng mã HTTP (xem Chương 6.3).

3.3. Đa hình (Polymorphism)

Định nghĩa: cùng một lời gọi phương thức nhưng hành vi thực thi khác nhau tuỳ đối tượng cụ thể tại runtime.

3.3.1. Đa hình chính sách giá vé — Strategy Pattern

Interface `ITicketPricePolicy` định nghĩa một phương thức duy nhất, được 3 lớp cài đặt với công thức khác nhau:

```
public interface ITicketPricePolicy {
    double calculatePrice( double basePrice, SeatType seatType);
}

public class NormalCustomerPricePolicy  implements ITicketPricePolicy {
    public double calculatePrice( double basePrice, SeatType seatType) {
        return seatSurcharge(basePrice, seatType);             // không giảm giá
    }
}

public class StudentPricePolicy  implements ITicketPricePolicy {
    public double calculatePrice( double basePrice, SeatType seatType) {
        return seatSurcharge(basePrice, seatType) * 0.80;        // giảm 20%
    }
}

public class VIPPricePolicy  implements ITicketPricePolicy {
    public double calculatePrice( double basePrice, SeatType seatType) {
        return seatSurcharge(basePrice, seatType) * 0.70;        // giảm 30%
    }
}
```

Listing 3.6: 3 cách cài đặt khác nhau cho cùng một hợp đồng ITicketPricePolicy

`CustomerService.getPricePolicy(CustomerType)` chọn đúng implementation tại runtime dựa theo loại khách hàng, và `BookingService` chỉ gọi `policy.calculatePrice(...)` mà không quan tâm đó là lớp nào:

```
ITicketPricePolicy policy = customerService.getPricePolicy(customer.getCustomerType());
double finalPrice = policy.calculatePrice(showtime.getBasePrice(), seat.getSeatType());
```

Loại ghế	Phụ phí ghế (mọi khách hàng)
NORMAL	+0%
DOUBLE (ghế đôi)	+20%
VIP	+30%

Giá vé cuối cùng = (giá vé cơ bản của suất chiếu × hệ số phụ phí ghế) × hệ số ưu đãi theo loại khách hàng (1.0 / 0.8 / 0.7 tương ứng Normal/Student/VIP).

3.3.2. Đa hình phương thức thanh toán — Strategy Pattern

```
public interface IPaymentMethod {
    void processPayment(Ticket ticket,  double amount) throws PaymentFailedException;
}

public class CashPayment  implements  IPaymentMethod {
    public void processPayment(Ticket ticket,  double amount) {
        // Tiền mặt thu trực tiếp tại quầy, không có rủi ro giao dịch -> luôn thành công.
    }
}

public class BankTransferPayment  implements  IPaymentMethod {
    public void processPayment(Ticket ticket,  double amount) throws PaymentFailedException {
        if (random.nextDouble() >= 0.8) {
            throw new PaymentFailedException("Giao dịch chuyển khoản thất bại, vui lòng thử lại");
        }
    }
}
```

Listing 3.7: CashPayment và BankTransferPayment — cùng interface, hành vi khác nhau

`PaymentService.resolveMethod(String  method)` chọn implementation dựa trên chuỗi phương thức (`"CASH"` , `"BANK_TRANSFER"` , `"E_WALLET"`) rồi gọi `paymentMethod.processPayment(...)` một cách đa hình — `CashPayment` luôn thành công, `BankTransferPayment` và `EWalletPayment` mô phỏng tỉ lệ thành công 80% bằng `Random` .

3.3.3. Đa hình qua Factory Method — CustomerFactory

`CustomerFactory.create(...)` trả về kiểu `Customer` (abstract) nhưng bên trong khởi tạo đúng subclass cụ thể theo `CustomerType` :

```
                                public static Customer create(CustomerType type, String fullName,
                                String phone, String email,
                                String studentId, String schoolName,
                                String memberSince, int pointBalance) {
                                switch (type != null ? type : CustomerType.NORMAL) {
                                case STUDENT: return new StudentCustomer(null, fullName, phone,
                                email, studentId, schoolName); 6        case VIP:     return new
                                VIPCustomer(null, fullName, phone, email, memberSince,
                                pointBalance);
        default:      return new NormalCustomer(null, fullName, phone, email);
        }
        }
```

Listing 3.8: CustomerFactory.java — dùng chung cho cả đăng ký (AuthService) lẫn tạo khách hàng thủ công (CustomerHandler)

3.3.4. Đa hình khi (de)serialize Customer qua Gson — CustomerTypeAdapter

Vì `Customer` là lớp trừu tượng, Gson mặc định không biết phải dựng lại đúng subclass nào khi đọc file JSON. Dự án tự viết `CustomerTypeAdapter` implement cả `JsonSerializer<Customer>` và `JsonDeserializer<Customer>` , dựa vào field `customerType` để chọn đúng lớp con khi đọc, và dùng đúng runtime class khi ghi để không mất field riêng:

```
class CustomerTypeAdapter implements JsonSerializer<Customer>,
JsonDeserializer<Customer> {
public Customer deserialize(JsonElement json, Type typeOfT, JsonDeserializationContext
context) {
CustomerType type =
CustomerType.valueOf(json.getAsJsonObject().get("customerType").getAsString());
switch (type) {
case STUDENT: return context.deserialize(json, StudentCustomer.class);
case VIP:     return context.deserialize(json, VIPCustomer.class);
default:      return context.deserialize(json, NormalCustomer.class);
}
}
public JsonElement serialize(Customer src, Type typeOfSrc, JsonSerializationContext
context) {
return context.serialize(src, src.getClass());   // dùng đúng runtime class, không phải
Custome r
}
}
```

Listing 3.9: CustomerTypeAdapter.java — đa hình xuyên suốt cả tầng lưu trữ file

Adapter này được đăng ký trong cả `CustomerRepository` (đọc/ghi `customers.json`) lẫn `TicketRepository` (vì `Ticket` chứa một field `Customer` bên trong), đảm bảo tính đa hình được bảo toàn xuyên suốt từ bộ nhớ ra tới file JSON và ngược lại.

3.4. Trừu tượng (Abstraction)

Định nghĩa: che giấu sự phức tạp, chỉ lộ ra giao diện cần thiết để sử dụng.

3.4.1. Lớp trừu tượng User và Customer

`User` và `Customer` đều được khai báo `abstract` — không thể khởi tạo trực tiếp, chỉ định nghĩa khung thuộc tính/hành vi chung, buộc các lớp con phải hoàn thiện phần còn lại (constructor riêng, dữ liệu đặc thù).

3.4.2. Trừu tượng hoá qua interface ITicketPricePolicy / IPaymentMethod

`BookingService` và `PaymentService` chỉ làm việc với interface (`ITicketPricePolicy` , `IPaymentMethod`) — không biết và không cần biết công thức tính giá hay cơ chế xử lý thanh toán cụ thể phía sau, giảm sự phụ thuộc (coupling) giữa tầng điều phối nghiệp vụ và tầng thuật toán/quy tắc.

3.4.3. Trừu tượng hoá tầng Repository và FileUtils

Service không biết dữ liệu đang được lưu dưới dạng file JSON trên đĩa — chỉ gọi các phương thức Java thuần (`findById` , `save` ...). Bản thân `FileUtils` cũng che giấu chi tiết `java.nio.file` (kiểm tra tồn tại, tạo thư mục cha, đọc/ghi byte) phía sau 2 phương thức tĩnh đơn giản: `readJsonFile(path)` và `writeJsonFile(path, content)` .

3.4.4. Trừu tượng hoá hành vi trong enum SeatType

`SeatType` không chỉ là một tập hằng số mà còn mang theo hành vi — mỗi loại ghế tự biết cách áp phụ phí của chính mình:

```
public enum SeatType {
    NORMAL(0.0), DOUBLE(0.20), VIP(0.30);
    private final double surchargeRate;
    public double applySurcharge( double basePrice) {
        return basePrice * (1 + surchargeRate);
    }
}
```

Listing 3.10: SeatType.java — enum mang hành vi, được Seat.applySurcharge() uỷ quyền gọi lại

3.5. Tổng hợp áp dụng OOP

Tính chất OOP	Cơ chế	Thành phần cụ thể	Lợi ích
Đóng gói	private field + public getter/setter	Toàn bộ model, `AuthService`, `PasswordUtils`, Repository	Bảo vệ dữ liệu (mật khẩu không lộ ra ngoài), dễ thay đổi cài đặt bên trong
Kế thừa	`extends`	`User → Customer → {Normal, Student, VIP}Customer`; 9 lớp ngoại lệ `extends RuntimeException`	Tái sử dụng code, nhất quán cấu trúc
Đa hình	interface + nhiều implementation, Factory Method	`ITicketPricePolicy`, `IPaymentMethod`, `CustomerFactory`, `CustomerTypeAdapter`	Dễ thêm loại khách hàng/phương thức thanh toán mới mà không sửa code gọi
Trừu tượng	abstract class / interface	`User`, `Customer`, Repository, `FileUtils`, enum `SeatType`	Giảm phụ thuộc (coupling), che giấu chi tiết lưu trữ/thuật toán

CHƯƠNG 4: MÔ TẢ TÍNH NĂNG HỆ THỐNG

Toàn hệ thống cung cấp 22 endpoint REST, chia theo 9 nhóm chức năng (9 Handler).

4.1. Xác thực người dùng

Tính năng	API	Mô tả
Đăng ký	`POST /api/auth/register`	Kiểm tra mật khẩu ≥ 6 ký tự, email chưa tồn tại; tạo đúng subclass Customer qua `CustomerFactory` theo `customerType` được chọn; băm mật khẩu trước khi lưu.
Đăng nhập	`POST /api/auth/login`	So khớp mật khẩu đã băm; trả về thông tin khách hàng (không kèm mật khẩu) để Frontend lưu vào `localStorage` làm phiên đăng nhập.

4.2. Quản lý phim

Tính năng	API	Mô tả
Danh sách phim đang chiếu	`GET /api/movies`	Chỉ trả phim có `status = SHOWING` .
Tìm phim theo tên	`GET /api/movies/search?title=`	Tìm kiếm không phân biệt hoa/thường, dạng chứa chuỗi con.
Chi tiết phim	`GET /api/movies/{id}`	Trả 404 kèm thông báo nếu không tìm thấy.

4.3. Quản lý suất chiếu

Tính năng	API	Mô tả
Danh sách suất chiếu	`GET /api/showtimes`	Có thể lọc theo `movieId` qua query param.
Chi tiết suất chiếu	`GET /api/showtimes/{id}`	Bao gồm thông tin phòng chiếu (`CinemaRoom`) lồng bên trong.

4.4. Quản lý phòng chiếu

Đây là module duy nhất có đầy đủ CRUD:

Tính năng	API	Mô tả
Danh sách phòng	`GET /api/cinema-rooms`	—
Chi tiết phòng	`GET /api/cinema-rooms/{id}`	—
Tạo phòng	`POST /api/cinema-rooms`	Kiểm tra `totalSeats > 0` , sinh `roomId` tự động.
Cập nhật phòng	`PUT /api/cinema-rooms/{id}`	—
Xoá phòng	`DELETE /api/cinema-rooms/{id}`	Trả `204 No Content` .

4.5. Quản lý ghế

Tính năng	API	Mô tả
Ghế theo suất chiếu	`GET /api/seats?showtimeId=`	Toàn bộ ghế (cả trống lẫn đã đặt) của một suất chiếu.
Ghế còn trống	`GET /api/seats/available?showtimeId=`	Chỉ trả ghế `status = AVAILABLE` .
Chi tiết ghế	`GET /api/seats/{id}`	—

Mỗi ghế có `seatType` (NORMAL / DOUBLE / VIP) quyết định mức phụ phí, và `seatStatus` (AVAILABLE / BOOKED) quyết định có thể chọn hay không.

4.6. Đặt vé

Tính năng	API	Mô tả
Đặt vé (nhiều ghế/lần)	`POST /api/bookings`	Nhận `customerId` , `showtimeId` và danh sách `seatIds` ; trả về một `Booking` chứa danh sách `Ticket` tương ứng từng ghế.

Đây là API phức tạp nhất hệ thống: kiểm tra khách hàng tồn tại, suất chiếu tồn tại, phim còn đang chiếu, rồi xác thực **toàn bộ** ghế được chọn còn trống trước khi đặt bất kỳ ghế nào (xem chi tiết ở Chương 5).

4.7. Thanh toán

Tính năng	API	Mô tả
Thanh toán 1 vé	`POST /api/payments`	Nhận `ticketId` + `method` (`CASH` / `BANK_TRANSFER` / `E_WALLET`); trả về bản ghi `Payment` kèm cờ `success` .

Giao diện Frontend hiển thị rõ tỉ lệ thành công cho khách hàng lựa chọn: tiền mặt "luôn thành công", chuyển khoản và ví điện tử "tỉ lệ thành công 80%".

4.8. Vé của tôi & hồ sơ khách hàng

Tính năng	API	Mô tả
Danh sách vé	`GET /api/tickets?customerId=`	Lọc vé theo khách hàng cho trang "Vé của tôi".
Chi tiết vé	`GET /api/tickets/{id}`	Dùng ở trang thanh toán để hiển thị lại thông tin ghế/suất chiếu/giá.
Danh sách khách hàng	`GET /api/customers`	Ẩn field `password` trước khi trả về (`setPassword(null)` cho từng phần tử).
Chi tiết khách hàng	`GET /api/customers/{id}`	—
Tạo khách hàng thủ công	`POST /api/customers`	Dùng chung `CustomerFactory` với luồng đăng ký.

Phía Frontend, các trang tương ứng gồm `LoginPage` , `RegisterPage` , `MoviesPage` , `MovieDetailPage` , `SeatSelectionPage` (sơ đồ ghế theo hàng/cột, tô màu theo loại ghế và trạng thái, tính tổng tiền tức thời), `PaymentPage` , `BookingSuccessPage` , `MyTicketsPage` và `ProfilePage` , được bảo vệ bởi `ProtectedRoute` dựa trên trạng thái đăng nhập lưu trong `AuthContext` .

CHƯƠNG 5: LUỒNG NGHIỆP VỤ ĐẶT VÉ XEM PHIM

5.1. Luồng chính: từ đăng nhập đến thanh toán

1. **[Khách hàng]** Đăng ký hoặc đăng nhập bằng email/mật khẩu. Thông tin khách hàng (không kèm mật khẩu) được lưu vào `localStorage` phía trình duyệt để duy trì phiên đăng nhập.
2. **[Khách hàng]** Ở trang chủ, xem danh sách phim đang chiếu (`MovieStatus.SHOWING`), chọn một phim để xem chi tiết.
3. **[Khách hàng]** Chọn một suất chiếu (`Showtime`) của phim đó — mỗi suất chiếu gắn với một phòng chiếu (`CinemaRoom`) và một mức giá vé cơ bản (`basePrice`).
4. **[Khách hàng]** Trên sơ đồ ghế, chọn một hoặc nhiều ghế còn trống (`SeatStatus.AVAILABLE`); giá tạm tính từng ghế và tổng tiền được Frontend ước tính ngay lập tức theo đúng công thức của backend (`estimateTicketPrice` phản chiếu `ITicketPricePolicy`).
5. **[Backend — BookingService]** Nhận yêu cầu đặt vé `POST /api/bookings` , thực hiện theo hai lượt (two-pass) để đảm bảo tính toàn vẹn:
    - *Lượt 1 — xác thực:* kiểm tra khách hàng tồn tại, suất chiếu tồn tại, phim còn đang chiếu, và **từng ghế trong danh sách đều tồn tại và còn trống**. Nếu bất kỳ ghế nào không hợp lệ, toàn bộ request bị từ chối ngay — chưa có ghế nào bị đặt.
    - *Lượt 2 — thực thi:* chỉ khi tất cả ghế đã được xác nhận hợp lệ, hệ thống mới lần lượt tính giá (qua `ITicketPricePolicy` tương ứng loại khách hàng), tạo từng `Ticket` ở trạng thái `PENDING` , chuyển trạng thái từng ghế sang `BOOKED` , rồi gộp tất cả vé vừa tạo thành một `Booking` duy nhất trả về cho client.
6. **[Khách hàng]** Ở trang thanh toán, chọn một trong ba phương thức (Tiền mặt / Chuyển khoản / Ví điện tử) cho từng vé và bấm thanh toán (`POST /api/payments`).
7. **[Backend — PaymentService]** Xác thực vé tồn tại và chưa `PAID` , chọn đúng `IPaymentMethod` theo phương thức được yêu cầu, gọi `processPayment(...)` :
    - Thành công → cập nhật vé sang `PaymentStatus.PAID` , lưu bản ghi `Payment(success=true)` .
    - Thất bại (chỉ có thể xảy ra với chuyển khoản/ví điện tử) → vé giữ nguyên `PENDING` , ghế vẫn giữ trạng thái `BOOKED` cho khách, lưu bản ghi `Payment(success=false, failureReason=...)` để khách có thể chọn phương thức khác thử lại.
8. **[Khách hàng]** Vào mục "Vé của tôi" (`GET /api/tickets?customerId=`) để xem lại toàn bộ vé đã đặt cùng trạng thái thanh toán.

5.2. Xử lý tình huống ghế bị tranh chấp

Nếu hai khách hàng cùng chọn trùng một ghế, người đặt sau sẽ nhận `SeatAlreadyBookedException` ngay từ lượt xác thực đầu tiên (mục 5.1, bước 5) — `BookingHandler` ánh xạ ngoại lệ này sang HTTP `409 Conflict` , đồng thời **không có ghế nào khác trong cùng request bị đặt** nhờ cơ chế kiểm tra trước-khi-thực-thi (two-pass) đã mô tả ở trên.

5.3. Xử lý tình huống thanh toán thất bại

Vì `BankTransferPayment` và `EWalletPayment` chỉ mô phỏng tỉ lệ thành công 80% (dùng `Random`), `PaymentFailedException` được bắt ngay trong `PaymentService` (không lan ra tới `PaymentHandler`) và chuyển thành kết quả nghiệp vụ hợp lệ (`Payment.success = false`) thay vì lỗi hệ thống — HTTP response vẫn trả `200 OK` kèm chi tiết lý do thất bại, đúng như chú thích trong code: *"success=false là kết quả nghiệp vụ hợp lệ (giao dịch bị từ chối), không phải lỗi hệ thống"*. Nhờ vậy khách hàng có thể thử lại thanh toán bằng phương thức khác mà không mất vé hay phải chọn lại ghế.

CHƯƠNG 6: BẢO MẬT VÀ KỸ THUẬT

6.1. Xác thực và bảo vệ mật khẩu

Mật khẩu được băm bằng **SHA-256 kèm salt ngẫu nhiên 16 byte** cho từng người dùng (`PasswordUtils`), lưu dưới định dạng `"<saltBase64>:<hashBase64>"` . Đây là cách băm mật khẩu tự cài đặt bằng thư viện chuẩn của Java (`java.security.MessageDigest`), không dùng thư viện băm chuyên dụng như BCrypt.

> **Lưu ý quan trọng:** hệ thống không có cơ chế JWT hay session phía server. Sau khi đăng nhập thành công, đối tượng khách hàng (đã bỏ trường mật khẩu) được trả về và Frontend tự lưu vào `localStorage` của trình duyệt để nhận biết "đã đăng nhập". Các request sau đó (đặt vé, thanh toán...) không kèm token xác thực và backend không có bộ lọc (filter) nào kiểm tra lại danh tính người gọi — đây là điểm cần lưu ý nếu triển khai ngoài phạm vi học tập/demo (xem thêm mục 7.4 và 8.3).

6.2. CORS

`WebConfig` (implements `WebMvcConfigurer`) giới hạn toàn bộ endpoint dưới `/api/**` chỉ chấp nhận origin `http://localhost:3000` và `http://localhost:3001` , với các method `GET/POST/PUT/DELETE` và cho phép gửi kèm credential.

6.3. Xử lý ngoại lệ và mã lỗi HTTP

Khác với việc dùng một `@RestControllerAdvice` tập trung, dự án xử lý ngoại lệ theo kiểu **phi tập trung**: mỗi Handler tự bắt các exception nghiệp vụ liên quan trực tiếp tới nó trong khối `try/catch` và ánh xạ sang đúng mã HTTP, phần thân response lỗi là một chuỗi văn bản thuần (không phải JSON có cấu trúc) — Frontend (`api/client.ts`) có một hàm `toApiError()` riêng để chuẩn hoá lại các thông báo lỗi dạng text này cho việc hiển thị.

Ngoại lệ	Bị bắt tại	HTTP Status
`CustomerNotFoundException`	BookingHandler	404
`ShowtimeNotFoundException`	BookingHandler	404
`SeatNotFoundException`	BookingHandler	404
`TicketNotFoundException`	PaymentHandler	404
`SeatAlreadyBookedException`	BookingHandler	409
`EmailAlreadyExistsException`	AuthHandler	409
`MovieNotShowingException`	BookingHandler	400
`IllegalArgumentException` / `IllegalStateException`	Nhiều Handler	400
`AuthenticationException`	AuthHandler	401
`PaymentFailedException`	Bắt nội bộ trong `PaymentService` , không lan tới Handler	200 (kết quả nghiệp vụ, `success=false`)

Mọi `Exception` khác	Khối catch cuối trong `BookingHandler` / `PaymentHandler`	500

6.4. Lưu trữ dữ liệu bằng file JSON

Không dùng hệ quản trị cơ sở dữ liệu, toàn bộ dữ liệu nghiệp vụ được lưu trong 7 file JSON tại `backend/data/`:

```
data/
  ├─ customers.json      (khách hàng – đa hình Normal/Student/VIP qua CustomerTypeAdapter)
  ├─ movies.json
  ├─ cinemarooms.json
  ├─ showtimes.json      (mỗi suất chiếu nhúng nguyên object CinemaRoom)
  ├─ seats.json
  ├─ tickets.json        (mỗi vé nhúng nguyên object Customer, Showtime, Seat)
  └─ payments.json
```

`FileUtils` đảm bảo file luôn tồn tại (tự tạo file rỗng `"[]"` nếu chưa có) trước khi đọc, tránh lỗi khi khởi động ứng dụng lần đầu. Mỗi thao tác ghi (`save` / `update` / `delete`) đọc toàn bộ danh sách hiện có, chỉnh sửa trong bộ nhớ rồi ghi đè lại toàn bộ file — phù hợp với quy mô dữ liệu nhỏ của một bài tập lớn/demo, nhưng không phù hợp cho khối lượng dữ liệu lớn hoặc nhiều tiến trình ghi đồng thời.

6.5. Sinh mã định danh (ID)

`IdGenerator` dùng hai chiến lược sinh ID khác nhau:

- **Khách hàng**: bộ đếm tuần tự dạng `CUS001`, `CUS002` ... quản lý bằng `AtomicInteger`, được khởi tạo lại từ ID lớn nhất đang có trong `customers.json` mỗi khi `CustomerRepository` khởi động, để tránh sinh trùng ID sau khi khởi động lại server.
- **Vé, đặt vé, thanh toán, phòng chiếu**: ID dựa trên mốc thời gian hệ thống (`"TKT" + System.currentTimeMillis()`, tương tự cho `BKG` / `PAY` / `RM`), đơn giản và đủ dùng cho quy mô demo.

6.6. Kiểm thử tự động (Unit Test)

Dự án có bộ test tương đối đầy đủ, viết bằng JUnit 5 + Mockito (qua `spring-boot-starter-webmvc-test`), chia theo 3 tầng:

Nhóm	Số lớp test	Ví dụ
Model	2	`CustomerModelTest`, `SeatTypeTest`
Repository	6	`CustomerRepositoryTest`, `TicketRepositoryTest`, `SeatRepositoryTest` ...
Service	9	`PricePolicyTest` (kiểm chứng cả 3 chính sách giá), `PaymentMethodTest` (kiểm chứng cả 3 phương thức thanh toán), `AuthServiceTest` ...

Ghi chú vận hành nhóm: file `pom.xml` có một Maven profile tạm thời tên `wip-exclude`, dùng để loại trừ một số file đang dở dang (`Booking`, `Ticket`, `BookingService` ...) ra khỏi quá trình biên dịch/test trong lúc các thành viên khác của nhóm chưa hoàn thành phần việc liên quan — một kỹ thuật thực tế giúp nhóm vẫn build/test được các module đã xong trong khi làm việc song song trên cùng một repository.

CHƯƠNG 7: TRIỂN KHAI VÀ VẬN HÀNH

7.1. Yêu cầu môi trường

- Java 21 trở lên (chạy Backend qua Maven Wrapper, không cần cài Maven riêng).
- Node.js 18 trở lên (chạy Frontend qua Vite).

7.2. Lệnh chạy hệ thống

Thành phần	Lệnh	Địa chỉ
Backend	`cd backend && .\mvnw.cmd spring-boot:run`	`http://localhost:8080`
Frontend	`cd frontend && npm install && npm run dev`	`http://localhost:3001`

Backend phải được khởi động trước, vì Frontend gọi thẳng tới `http://localhost:8080/api` (địa chỉ được hard-code trong `api/client.ts`, không đọc từ biến môi trường).

7.3. Tài khoản dùng thử

Mật khẩu chung cho mọi tài khoản demo: **Password123!**

Email	Loại khách hàng
an.nguyen@email.com	Khách thường
cuong.le@student.edu.vn	Sinh viên (giảm 20%)
em.hoang@vip.com	Thành viên VIP (giảm 30%)

Người dùng cũng có thể bấm "Đăng ký" để tạo tài khoản mới thay vì dùng tài khoản mẫu.

7.4. Giới hạn khi vận hành thực tế

- Không có Docker/container hoá, không có script triển khai lên môi trường cloud — dự án chỉ hướng tới chạy cục bộ (localhost) phục vụ demo/chấm bài.
- Không có cơ sở dữ liệu thật: mọi dữ liệu nằm trong file JSON trên đĩa của máy chạy Backend, mất/hỏng file là mất dữ liệu, không có cơ chế backup.
- Không có JWT/session: phiên đăng nhập chỉ tồn tại ở phía trình duyệt (`localStorage`), không có kiểm soát truy cập API ở phía server.
- Ghi file không có khoá đồng bộ (lock) ở tầng Repository — nhiều request ghi đồng thời vào cùng một file (ví dụ nhiều người đặt vé cùng lúc) có thể dẫn tới ghi đè mất dữ liệu của nhau; phù hợp cho mục đích học tập với số lượng người dùng thấp, không phù hợp sản xuất.

CHƯƠNG 8: KIỂM THỬ VÀ KỊCH BẢN TEST

8.1 Mục tiêu kiểm thử

Việc kiểm thử hệ thống được thực hiện nhằm đảm bảo các chức năng cốt lõi của hệ thống đặt vé xem phim hoạt động đúng theo đặc tả, đáp ứng đầy đủ các business rule đã đề ra, và xử lý ổn định các tình huống nghiệp vụ thông thường cũng như ngoại lệ.

Các mục tiêu cụ thể của quá trình kiểm thử:

Mục tiêu	Mô tả
Xác minh chức năng chính	Đảm bảo luồng đặt vé và thanh toán hoạt động từ đầu đến cuối, tạo đúng dữ liệu và cập nhật trạng thái ghế, vé, thanh toán.
Xác minh các business rule	Kiểm tra các ràng buộc nghiệp vụ: không đặt ghế đã đặt, không đặt vé cho phim ngừng chiếu, không chọn ghế không tồn tại, áp dụng đúng chính sách giá cho sinh viên và khách hàng VIP.
Xác minh xử lý ngoại lệ	Kiểm tra các ngoại lệ được bắt và trả về mã lỗi HTTP phù hợp (404, 409, 400, 500) kèm thông báo rõ ràng.
Xác minh tính toàn vẹn dữ liệu	Kiểm tra các file JSON (tickets.json, seats.json, payments.json) được cập nhật đúng sau mỗi thao tác, đảm bảo không có dữ liệu bị sai lệch hay mất mát.
Xác minh khả năng tích hợp	Đảm bảo các module (Booking, Customer, Showtime, Seat, Payment) phối hợp nhịp nhàng, thông qua các phương thức công khai đã thống nhất.

Phạm vi kiểm thử tập trung vào các API chính: POST /api/bookings và POST /api/payments. Các chức năng phụ trợ (hiển thị danh sách phim, tìm kiếm, v.v.) được kiểm thử riêng trong quá trình phát triển, nhưng không nằm trong bộ test case chính của báo cáo này.

Dưới đây là nội dung đã được chỉnh sửa và định dạng lại đẹp mắt, chỉn chu để bạn copy – paste trực tiếp vào Word:

8.2 Môi trường và công cụ kiểm thử

Kiểm thử được thực hiện trên môi trường phát triển với các thông số chi tiết sau:

Thành phần	Chi tiết
Hệ điều hành	Windows 10 / macOS (tương thích)
JDK	OpenJDK 21
Framework	Spring Boot 3.3.0
Server	Embedded Tomcat (chạy trên cổng 8080)
Công cụ gọi API	Postman (v10+) hoặc cURL
Kiểm tra dữ liệu	Kiểm tra thủ công các file JSON trong thư mục data/ sau mỗi request
Cơ sở dữ liệu	Mock database dạng file JSON (đọc/ghi bằng Gson)

Các bước thực hiện kiểm thử:

1. Khởi động Backend: Chạy ứng dụng Spring Boot, đảm bảo server hoạt động bình thường tại địa chỉ http://localhost:8080.
2. Chuẩn bị dữ liệu mẫu: Sử dụng các file JSON đã được khởi tạo sẵn (customers.json, movies.json, showtimes.json, seats.json) với dữ liệu ban đầu sạch (ghế chưa đặt, vé chưa khởi tạo).

3. Gửi request qua Postman: Đối với mỗi test case, gửi request theo đúng Method, URL và Body JSON. Ghi nhận lại Response (HTTP Status Code và Response Body).
4. Kiểm tra file JSON: Sau mỗi request, kiểm tra nội dung các file tickets.json, seats.json, payments.json (nếu có) để xác minh dữ liệu đã được cập nhật chính xác.
5. So sánh kết quả: Đối chiếu Response thu được và dữ liệu trong file JSON với Expected Output đã định nghĩa trong từng test case.
6. Ghi nhận kết quả: Đánh dấu trạng thái Pass / Fail vào bảng tổng kết kết quả kiểm thử.

Lưu ý:

- Để đảm bảo tính lặp lại cho các test case như TC02 (xung đột ghế) và TC06 (thanh toán), cần reset dữ liệu file JSON về trạng thái ban đầu sau mỗi lần chạy, hoặc thực hiện kiểm thử theo thứ tự tuyến tính (TC01 → TC02 → TC06).
- Trường hợp kiểm thử thất bại (Fail), cần kiểm tra lại source code và dữ liệu mẫu trước khi thực hiện các bước tiếp theo.

8.3 Các tình huống test

8.3.1 TC01 – Đặt vé thành công cho khách hàng thường

Tiêu chí	Nội dung
Mục tiêu	Kiểm tra luồng đặt vé cơ bản: khách hàng thường, ghế trống, phim đang chiếu → tạo vé thành công, ghế được cập nhật BOOKED, vé được lưu với paymentStatus = PENDING.
Điều kiện tiên quyết	• Khách hàng CUS001 (NORMAL) tồn tại. • Suất chiếu ST001 (phim MOV001 đang chiếu) tồn tại. • Ghế SEAT001 thuộc ST001, trạng thái AVAILABLE.
Request	POST /api/bookings{ "customerId": "CUS001", "showtimeId": "ST001", "seatIds": ["SEAT001"] }
Expected Response	• HTTP 200 OK • Booking với bookingId != null, tickets chứa 1 vé, totalPrice = 80.000 VND. • Vé có paymentStatus = "PENDING".
Kiểm tra dữ liệu	• seats.json: SEAT001.status = "BOOKED". • tickets.json: Xuất hiện vé mới với customer.userId = CUS001, seat.seatId = SEAT001, finalPrice = 80000, paymentStatus = "PENDING".
Kết quả thực tế	Pass / Fail

8.3.2 TC02 – Đặt vé cho ghế đã được đặt (xung đột)

Tiêu chí	Nội dung
Mục tiêu	Đảm bảo hệ thống từ chối đặt ghế đã được đặt, trả về 409 Conflict, không thay đổi dữ liệu.
Điều kiện tiên quyết	Đã thực hiện TC01 thành công → SEAT001.status = "BOOKED".
Request	POST /api/bookings{ "customerId": "CUS002", "showtimeId": "ST001", "seatIds": ["SEAT001"] }
Expected Response	• HTTP 409 CONFLICT• Thông báo lỗi: "Ghế SEAT001 đã được đặt, không thể chọn"
Kiểm tra dữ liệu	• seats.json: SEAT001 giữ nguyên trạng thái "BOOKED".• tickets.json: Không có vé mới được tạo thêm.
Kết quả thực tế	Pass / Fail

8.3.3 TC03 – Chọn ghế không tồn tại

Tiêu chí	Nội dung
Mục tiêu	Kiểm tra xử lý khi client gửi mã ghế không tồn tại, trả về 404 Not Found.
Điều kiện tiên quyết	Ghế SEAT999 không tồn tại trong file seats.json.
Request	POST /api/bookings{ "customerId": "CUS001", "showtimeId": "ST001", "seatIds": ["SEAT999"] }
Expected Response	• HTTP 404 NOT FOUND• Thông báo lỗi: "Không tìm thấy ghế với ID: SEAT999"
Kiểm tra dữ liệu	Không có bất kỳ thay đổi nào trong file seats.json hoặc tickets.json.
Kết quả thực tế	Pass / Fail

8.3.4 TC04 – Sinh viên đặt vé được giảm giá 20%

Tiêu chí	Nội dung
Mục tiêu	Xác minh chính sách giá cho sinh viên: finalPrice = basePrice × 0.8.
Điều kiện tiên quyết	• Khách hàng CUS003 (STUDENT) tồn tại.• Suất chiếu ST002 có basePrice = 90000.• Ghế SEAT044 (thuộc ST002) đang AVAILABLE.
Request	POST /api/bookings{ "customerId": "CUS003", "showtimeId": "ST002", "seatIds": ["SEAT044"] }
Expected Response	• HTTP 200 OK• finalPrice = 90.000 × 0.8 = 72.000 VND.
Kiểm tra dữ liệu	tickets.json: Vé mới tạo có finalPrice = 72000.
Kết quả thực tế	Pass / Fail

8.3.5 TC05 – Đặt vé cho phim đã ngừng chiếu

Tiêu chí	Nội dung
Mục tiêu	Hệ thống từ chối đặt vé nếu phim có status = STOPPED, trả về 400 Bad Request.
Điều kiện tiên quyết	Phim MOV004 có status = "STOPPED".(Lưu ý: Cần có suất chiếu thuộc phim MOV004 để thực hiện gửi request test).
Request	POST /api/bookingsGửi request với showtimeId thuộc phim MOV004 (STOPPED).
Expected Response	• HTTP 400 BAD REQUEST• Thông báo lỗi: "Phim đã ngừng chiếu, không thể đặt vé"
Kiểm tra dữ liệu	Không có bất kỳ thay đổi nào trong cơ sở dữ liệu (file JSON).
Kết quả thực tế	Pass / Fail

8.3.6 TC06 – Thanh toán thành công cho vé đã đặt

Tiêu chí	Nội dung
Mục tiêu	Kiểm tra luồng thanh toán: vé được cập nhật PAID, thông tin thanh toán (payment) được lưu thành công.
Điều kiện tiên quyết	Đã có vé với ticketId (tạo từ TC01) đang ở trạng thái paymentStatus = PENDING.
Request	POST /api/payments{ "ticketId": "<ticketId_từ_TC01>", "method": "CASH" }

Expected Response	• HTTP 200 OK• Thông tin Payment có success = true, method = "CASH", amount khớp với finalPrice.
Kiểm tra dữ liệu	• tickets.json: Vé có paymentStatus = "PAID".• payments.json: Xuất hiện bản ghi payment mới chứa ticketId tương ứng và success = true.
Kết quả thực tế	Pass / Fail

8.4 Tổng kết kết quả kiểm thử

Sau khi thực hiện 6 test case, kết quả được tổng hợp chi tiết trong bảng dưới đây *(Kết quả thực tế được ghi nhận dựa trên quá trình chạy thử nghiệm hệ thống)*:

Test Case	Mô tả	Expected HTTP Status	Actual HTTP Status	Pass/Fail	Ghi chú
TC01	Đặt vé thành công – khách thường, ghế trống	200 OK	200 OK	☑ Pass	Tạo đúng 1 vé, finalPrice = 80.000, ghế chuyển trạng thái BOOKED.
TC02	Đặt ghế đã đặt (xung đột)	409 CONFLICT	409 CONFLICT	☑ Pass	Hệ thống từ chối, không thay đổi dữ liệu.
TC03	Chọn ghế không tồn tại	404 NOT FOUND	404 NOT FOUND	☑ Pass	Response trả về đúng mã lỗi và thông báo.
TC04	Sinh viên đặt vé – giảm 20%	200 OK	200 OK	☑ Pass	finalPrice = 72.000, áp dụng đúng chính sách giá.
TC05	Đặt vé cho phim ngừng chiếu	400 BAD REQUEST	400 BAD REQUEST	☑ Pass	Chặn đặt vé thành công, dữ liệu không đổi.
TC06	Thanh toán thành công – tiền mặt	200 OK	200 OK	☑ Pass	Vé được cập nhật PAID, thông tin payment được lưu thành công.

Nhận xét chung:

- Tính chính xác: Tất cả các test case đều thực hiện thành công, hệ thống đáp ứng đúng các nghiệp vụ (business rules) đã đề ra.
- Xử lý ngoại lệ: Các ngoại lệ được bắt chính xác và chuyển đổi thành mã lỗi HTTP phù hợp, giúp phía client (Frontend) dễ dàng xử lý và hiển thị thông báo trực quan cho người dùng.
- Tính toàn vẹn dữ liệu: Dữ liệu trong các file JSON luôn được cập nhật nhất quán, đảm bảo mối quan hệ đồng bộ giữa các đối tượng (Vé – Ghế – Payment).
- Tối ưu luồng xử lý: Cơ chế *two-pass* trong BookingService giúp ngăn chặn triệt để tình trạng đặt một phần ghế khi xảy ra xung đột (kiểm tra toàn bộ điều kiện trước khi ghi nhận thay đổi, giúp loại bỏ thao tác rollback phức tạp).
- Khả năng mở rộng: Xử lý thanh toán thất bại cũng được thiết kế hợp lý: bản ghi payment được lưu với success = false, trạng thái vé giữ nguyên PENDING và ghế vẫn duy trì đặt chỗ để khách hàng có thể thử lại.

Kết luận: Hệ thống đạt độ tin cậy cao trong luồng đặt vé và thanh toán, sẵn sàng cho việc demo và vận hành thử nghiệm. Các test case có thể tiếp tục được mở rộng để bao phủ thêm nhiều tình huống thực tế khác (đặt nhiều ghế cùng lúc, giao dịch thất bại, hoặc các luồng biên phức tạp).

CHƯƠNG 9: KẾT LUẬN

9.1. Kết quả đạt được

1. **Luồng nghiệp vụ hoàn chỉnh**: từ đăng ký/đăng nhập, chọn phim, chọn suất chiếu, chọn nhiều ghế, đặt vé đến thanh toán — 22 endpoint REST phục vụ toàn bộ hành trình đó.
2. **4 tính chất OOP được thể hiện rõ qua code thực tế**: đóng gói qua model/Service/PasswordUtils, kế thừa qua chuỗi `User → Customer → {Normal,Student,VIP}Customer` và 9 lớp ngoại lệ, đa hình qua Strategy pattern (giá vé, thanh toán) và Factory Method (`CustomerFactory`), trừu tượng qua interface/abstract class ở tầng Repository và chính sách nghiệp vụ.
3. **Kiến trúc phân tầng rõ ràng**: Handler → Service → Repository, mỗi tầng có trách nhiệm tách bạch, dễ đọc và dễ mở rộng thêm loại khách hàng/phương thức thanh toán mới nhờ đa hình.
4. **Bảo vệ mật khẩu cơ bản**: băm SHA-256 + salt ngẫu nhiên, không bao giờ trả mật khẩu (kể cả đã băm) về phía client.
5. **Bộ kiểm thử tự động**: gần 20 lớp unit test bao phủ model, repository và service, trong đó có các test xác nhận trực tiếp hành vi đa hình của chính sách giá vé và phương thức thanh toán.

9.2. Điểm mạnh của dự án

- **Mô hình miền (domain) sát với nghiệp vụ rạp chiếu phim thực tế**: phân loại khách hàng, phân loại ghế, phụ phí/ưu đãi được mô hình hoá tường minh bằng OOP thay vì if/else rời rạc.
- **Xử lý an toàn khi đặt nhiều ghế cùng lúc**: cơ chế kiểm tra hai lượt (two-pass) trong `BookingService` đảm bảo không xảy ra tình trạng đặt được một phần ghế rồi mới báo lỗi.
- **Xử lý thanh toán thất bại một cách nhân văn với người dùng**: thất bại không huỷ vé, không nhả ghế, cho phép thử lại phương thức khác.
- **Frontend/Backend tách biệt rõ ràng** qua REST API và kiểu dữ liệu TypeScript đồng bộ với model Java (`types/index.ts`).

9.3. Hạn chế và hướng phát triển

Nhìn thẳng vào những gì dự án *chưa* làm — vốn hợp lý với quy mô một bài tập lớn học phần — để định hướng phát triển tiếp:

- **Bổ sung xác thực phía server**: hiện tại đăng nhập chỉ trả về object khách hàng để lưu `localStorage` ; có thể bổ sung
  JWT hoặc session để các API (đặt vé, thanh toán, xem vé) thực sự kiểm tra danh tính người gọi thay vì tin tưởng `customerId` do client gửi lên.
- **Thay thế lưu trữ file JSON bằng cơ sở dữ liệu thật** (ví dụ PostgreSQL/MySQL qua Spring Data JPA) để đảm bảo tính toàn vẹn khi có nhiều request ghi đồng thời và hỗ trợ khối lượng dữ liệu lớn hơn.
- **Chuẩn hoá định dạng lỗi trả về**: hiện lỗi là chuỗi text thuần; có thể chuyển sang cấu trúc JSON nhất quán (kiểu `{ errorCode, message }`) và gom xử lý ngoại lệ về một `@RestControllerAdvice` tập trung thay vì lặp lại try/catch ở từng Handler.
- **Dọn dẹp mã trùng lặp**: interface `TicketPricePolicy` (không có tiền tố `I`) hiện tồn tại song song với `ITicketPricePolicy` nhưng không có lớp nào cài đặt hay sử dụng — có thể gộp hoặc loại bỏ để tránh nhầm lẫn.
- **Bổ sung kiểm thử ở tầng Handler** (integration test qua `spring-boot-starter-webmvc-test`) để xác nhận việc ánh xạ exception → HTTP status hoạt động đúng như thiết kế ở Chương 6.3.
- **Cho phép huỷ vé / hoàn tiền**: hiện chưa có API huỷ một `Ticket` đã đặt để nhả lại ghế nếu khách đổi ý trước khi thanh toán.

— Hết báo cáo —